

Relatório Final — Sistema de Agendamento de Consultas (G8)

Autor: Alberto de Rezende Carneiro Tomaz — **líder do grupo**

Disciplina: Programação Orientada a Objetos (POO)

Repositório: poo-ec-2026-1/g8 · **Stack:** Java 17 + JavaFX 17 + SQLite (ORMLite) + JUnit 5 / TestFX

1. Do que se trata o projeto

O sistema é uma aplicação **desktop de agendamento de consultas para uma clínica**. Uma **secretária** gerencia pacientes, médicos e a agenda; cada **agendamento** vincula um médico e um paciente a uma data/horário, com bloqueio de conflito de horário; cada paciente tem um **prontuário** com o histórico dos médicos que o atenderam. O acesso é por perfil: a **secretária** entra só com CPF; o **médico** entra com CPF + senha. A persistência é local (SQLite), e o primeiro acesso é destravado por um *seed* idempotente que cria uma secretária administradora.

A entrega desta etapa foi menos sobre “adicionar features” e mais sobre **transformar um protótipo que não executava em um sistema que roda, é validado e é testável** — e é isso que este relatório defende.

2. Meu papel e o que eu fiz

Atuei como **líder e desenvolvedor principal**, cobrindo quatro frentes: arquitetura/estrutura, validação e regras de negócio, front-end/UX e documentação/UML, além de coordenar o fluxo de Git do grupo. Dois blocos de trabalho foram de colegas e eu preservo o crédito deles: o utilitário de validação de CPF (*ValidadorUtils*, base do Gabriel) e a **reformulação TDD** da suíte + os **testes de UI (TestFX)** e o relatório de QA (Mateus). Meu trabalho se apoiou nessas bases e as integrou ao restante do sistema.

O ponto de partida tinha um problema grave: **o aplicativo não compilava/executava** na primeira avaliação (nota 7,2/10; fraquezas em Precisão, Profundidade, Volume/UML e Criatividade). O foco foi atacar exatamente esses pontos.

3. Arquitetura e decisões de POO (o núcleo do trabalho)

Reorganizei o código em **camadas por responsabilidade**, em pacotes *application*, *model*, *repository*, *controller* e *util* — um estilo MVC. Sobre isso, algumas decisões que considero as mais importantes:

Repository Pattern. Cada entidade tem um **Repository* que encapsula o DAO do ORMLite. A persistência fica isolada do domínio: os controllers falam com repositórios, não

com o banco. Isso é baixo acoplamento na prática — trocar o mecanismo de persistência não toca no domínio nem na UI.

Injeção de dependência por construtor. O `MainApp` é o “compositor”: ele cria a conexão (`Database`), instancia os cinco repositórios **uma vez** e os injeta nos controllers e no `DatabaseSeeder`. Nenhuma classe cria a própria dependência de banco. Além de reduzir acoplamento, é o que torna o sistema **testável** — nos testes, injeta-se um banco `:memory:` no lugar do `hospital.db`.

Validação em duas camadas (defesa em profundidade). As invariantes de domínio vivem **no construtor da entidade** (CPF, data, campos obrigatórios lançam `IllegalArgumentException`), e são **revalidadas no repositório** antes de persistir. O construtor vazio permissivo exigido pelo ORMLite é preservado, mas o construtor “de verdade” nunca deixa nascer um objeto inválido. A entidade se protege sozinha, independentemente de quem a chama.

Falhas explícitas em vez de silenciosas. A Agenda (e a camada de repositório) **lança exceção** em erros de regra — horário ocupado, dados inválidos — em vez de só imprimir no console. Isso permite que a UI **reaja** (mostrar mensagem vermelha, bloquear a ação). Um erro que ninguém percebe é pior do que um erro que estoura.

SRP na prática. A lógica de *seed* foi **extraída** do `MainApp` para uma classe dedicada, `DatabaseSeeder`, que só cria a secretária inicial se o banco estiver vazio (idempotente). O `MainApp` voltou a ter uma responsabilidade só: iniciar e navegar.

4. Insights que achei mais interessantes

Se eu tivesse que escolher os três aprendizados que mais me marcaram, seriam estes:

(a) Acoplamento inevitável, isolado num só lugar — `CadastroValidator`. O grupo decidiu que o CPF é **único globalmente**: o mesmo CPF não pode existir como Cliente, Médico e Secretária ao mesmo tempo. Essa regra é, por natureza, acoplada — ela precisa cruzar as três tabelas de usuário. Em vez de espalhar esse acoplamento dentro de cada repositório, criei a classe `CadastroValidator.garantirCpfDisponivel(cpf)`, que concentra a regra num único ponto e é chamada no `create` dos três repositórios. O insight: **quando o acoplamento é inerente ao requisito, o objetivo não é eliminá-lo, é localizá-lo** — deixá-lo numa classe com nome próprio em vez de difuso pelo sistema.

(b) Uma Scene, troca só o root. Na versão anterior, cada navegação criava um `new Scene(...)`, o que **resetava o tamanho da janela e saía do modo tela cheia** a cada tela. Passei o `MainApp` a manter **uma única Scene** e trocar apenas o root (`scene.setRoot(novaView)`). Detalhe pequeno de implementação, impacto grande de usabilidade. Aprendi que UX às vezes mora numa decisão de arquitetura, não num CSS.

(c) Padrão de máscara reutilizável — e saber quando não abstrair. Isolei as máscaras de entrada (`MascaraCpf`, `MascaraData`, `MascaraHorario`) como `TextFormatter`

reaproveitáveis, tirando a formatação de dentro dos controllers. As três são quase idênticas e caberia uma única classe parametrizada — decidi **manter o padrão “uma classe por máscara” já existente** para não introduzir uma convenção nova numa entrega de fechamento. Reconhecer a duplicação e escolher conscientemente conviver com ela (com o motivo anotado) foi, pra mim, tão importante quanto o código em si.

5. Front-end e UX

O trabalho de front-end desta etapa concentrou-se em **tirar estilo e lógica de formatação de dentro do código Java**:

- **CSS externo (`styles.css`).** Tipografia, cores e botões que antes eram `Font.font(...)` / `setStyle(...)` *inline* em cada controller passaram para uma folha de estilo única, carregada no `MainApp`. Os controllers agora usam `styleClass` (`titulo-tela`, `botao-verde`, `status-erro/status-sucesso`), e as mensagens de status alternam entre classes em vez de setar cor fixa em Java.
- **Feedback visual de erro/sucesso.** Um helper `estilo(label, msg, ok)` troca a classe da mensagem — vermelho para erro, verde para sucesso — reaproveitando a validação já existente.
- **Dashboard adaptado ao perfil.** O `DashboardController` monta botões diferentes para Secretária e Médico, seguindo o diagrama de casos de uso.
- **Atalhos de teclado (Enter confirma).** Extraí a ação dos formulários para um `EventHandler` reutilizável e liguei ao `setOnAction` dos campos, para que Enter confirme igual ao clique no botão (Login, Agendar, Cadastro, Cancelar).

Sou honesto sobre o limite: o **CSS externo está pronto, mas o dashboard com métricas** (consultas do dia, contagens) e o feedback em tempo real mais rico **ainda não foram implementados** — é o próximo item do roadmap (ver Seção 8).

6. Documentação e UML

Refiz o `docs/uml/` com fontes `.puml` **revisadas contra o código** (não contra a documentação antiga, que estava desatualizada): diagramas de pacotes, classes de domínio, casos de uso e sete diagramas de sequência (login, cadastro, agendar, cancelar, acessar/atualizar prontuário, registrar atendimento). Duas telas que **faltavam por completo** na documentação antiga — “Visualizar Agenda” e “Atualizar Prontuário” — foram incluídas. Cada `.puml` tem um `.svg` renderizado ao lado, no mesmo estilo desenhado à mão dos existentes. Também mantenho o `ESTADO_ATUAL.md`, um handoff técnico que descreve arquitetura, decisões, pendências e roadmap — a “fonte de verdade” que permite outra pessoa (ou outra sessão) retomar o projeto sem contexto perdido.

Um princípio que segui: **quando a documentação diverge do código, o código vence** — e a divergência é registrada, não escondida.

7. Qualidade e processo

O projeto é **TDD-restrito**: a suíte exige o comportamento ideal, não documenta bugs. A reformulação TDD e os testes de UI (TestFX) são do Mateus; meu trabalho foi manter a suíte **verde** ao evoluir o sistema e **adicionar testes ao mudar comportamento** — por exemplo, ao introduzir a validação de horário na persistência e o CPF único global, entraram testes cobrindo horário inválido recusado e CPF duplicado (mesmo tipo e entre tipos diferentes), usando banco `:memory:`. Fechamento atual: **62 testes, 0 falhas**.

No processo de Git, coordenei o fluxo do grupo: **branch por funcionalidade + PR, main sempre verde** (nada é integrado com teste vermelho), commits pequenos e temáticos. Um cuidado real de equipe: ao integrar código de terceiros, **preservar a autoria original** (ex.: o `ValidadorUtils` do Gabriel entrou com a autoria dele).

8. Limitações e trabalhos futuros (o que eu mesmo critico)

Sendo crítico com a entrega, estes são os pontos que eu abriria como dívida técnica:

1. **View fundida ao controller (sem FXML).** Hoje a tela é construída em código Java dentro de cada controller — não existe pacote view. Funciona, mas mistura responsabilidades. Migrar para **FXML** separaria view de lógica de verdade. É a maior dívida arquitetural em aberto.
2. **Regra de conflito de horário duplicada.** O `AgendamentoController` reimplementa a checagem de conflito *inline* (`loadAll().stream().anyMatch(...)`), enquanto a mesma regra já existe em `Agenda.novaConsulta()`. Regra de negócio em dois lugares = risco de divergência. O certo seria a UI delegar para o domínio e o repositório expor uma busca filtrada em vez de `loadAll()`.
3. **Busca de CPF inconsistente.** O `AgendaController` compara CPF de forma **exata** (`equals`), enquanto o `AtualizarProntuarioController` compara **normalizado**. É a mesma regra (“achar cliente por CPF”) implementada de duas formas — deveria ser uma só.
4. **Login da secretária não checa senha.** O model tem o campo, mas o `LoginController` valida só o CPF dela.
5. **Sem ciclo de status da consulta.** Cancelar hoje **exclui** a Consulta; não há transição “Agendada → Realizada → Cancelada” porque a entidade não tem campo de status. É um requisito de domínio para a próxima etapa (e exigirá re-atualizar o UML).
6. **Observações do atendimento não são salvas.** `ProntuarioMedico` (ponte N:N) não tem

campos de data/anotações, então o texto digitado no registro de atendimento se perde.

7. **Dashboard com métricas + feedback em tempo real** ainda pendente (Seção 5).

9. Considerações finais

O eixo desta entrega foi **maduridade de engenharia, não volume de telas**: fazer o sistema executar, blindá-lo com validação em duas camadas, isolar as regras acopladas em lugares nomeados, deixar as falhas explícitas e apoiar tudo numa suíte de testes verde e num UML que corresponde ao código. As limitações que listei não são surpresas descobertas na correção — são **decisões conscientes de escopo**, documentadas para quem continuar. Para mim, o aprendizado mais valioso de POO neste projeto não foi um padrão específico, e sim a disciplina de **decidir onde uma responsabilidade mora** — e conseguir justificar cada escolha.